Scenario:

The problem is to create an online interrogation panel application using the Model-View-Controller (MVC) architecture that allows interogators to create problems with four choices and solution through a form. The problems should be stored in a database and users should be able to see the problems, and it's appropriate solution for each problem in a list view

Exercise Steps:

1. Create a new ASP.NET Core MVC project in Visual Studio.
2. Add a new controller called **"InterrogationController".**
3. Add a new folder called **"Models"** in the project and create a class called **"Problem.cs"** with the following properties with data annotations.

```
using System.ComponentModel.DataAnnotations;

namespace ASPNET6_Template.Models
{
    8 references
    public class Problem
    {
        [Key]
        0 references
        public int Id { get; set; }
        [Required(ErrorMessage = "Please provide Problem")]
        5 references
        public string InterrogatorsProblem { get; set; }
        [Required(ErrorMessage = "Please provide Choice1")]
        5 references
        public string Choice1 { get; set; }
        [Required(ErrorMessage = "Please provide Choice2")]
        5 references
        public string Choice2 { get; set; }
        [Required(ErrorMessage = "Please provide Choice3")]
        5 references
        public string Choice3 { get; set; }
        [Required(ErrorMessage = "Please provide Choice4")]
        5 references
        public string Choice4 { get; set; }
        [Required(ErrorMessage = "Please provide Solution")]
        5 references
        public string Solution { get; set; }
    }
}
```

4. Install these mandatory packages and create database before creating the views

5. Add a new folder called **"Data"** in the project and create a class called **"InterrogationDbContext.cs"** which inherits **DbContext** class.

6. In appsettings.json, include tis connection string configuration,

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DBConnection": "server=(LocalDB)\\MSSQLLocalDB;database=InterrogationPanelDB;Trusted_Connection=true;"
  }
}
```

7. Let us create Dbset property in the **InterrogationDbContext** class.

```
using ASPNET6_Template.Models;
using Microsoft.EntityFrameworkCore;

namespace ASPNET6_Template.Data
{
    7 references
    public class InterrogationDbContext : DbContext
    {
        0 references
        public InterrogationDbContext(DbContextOptions<InterrogationDbContext> options) : base(options)
        {

        }

        2 references
        public DbSet<Problem> Problems { set; get; }
    }
}
```

8. In Class **Program.cs** initialize the sql server and mention the database connection, which exists in the appsettings.json file.

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<InterrogationDbContext>
    (options => options.UseSqlServer
    (builder.Configuration.GetConnectionString("DBConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days.
    // You may want to change this for production scenarios,
    // see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Interrogation}/{action=Catalog}/{id?}");

app.Run();
```

9. Open Package manager console to create migration in order to create database.

```
Package source: All                    Default project: ASPNET6_Template
Each package is licensed to you by its owner. NuGet is not responsible for, no
additional licenses. Follow the package source (feed) URL to determine any dep

Package Manager Console Host Version 6.2.1.2

Type 'get-help NuGet' to see all available NuGet commands.

PM> add-migration sampleDB
```

10. When the build is succeeded then update database by this below comment,

```
Package source: All                    Default project: ASPNET6_Template
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
```

11. You have successfully created the database. There is a new folder named **Migrations** available in your project when the migration is successful.

12. Let us create required action methods in the **InterrogationController**. Then use dependency injection of the class **InterrogationDbContext** in the **InterrogationController**.

```
using ASPNET6_Template.Data;
using ASPNET6_Template.Models;
using Microsoft.AspNetCore.Mvc;

namespace ASPNET6_Template.Controllers
{
    1 reference
    public class InterrogationController : Controller
    {
        InterrogationDbContext db;

        0 references
        public InterrogationController(InterrogationDbContext _db)
        {
            db = _db;
        }

        [HttpGet]
        0 references
        public ActionResult Craft()
        {
            return View();
        }
```

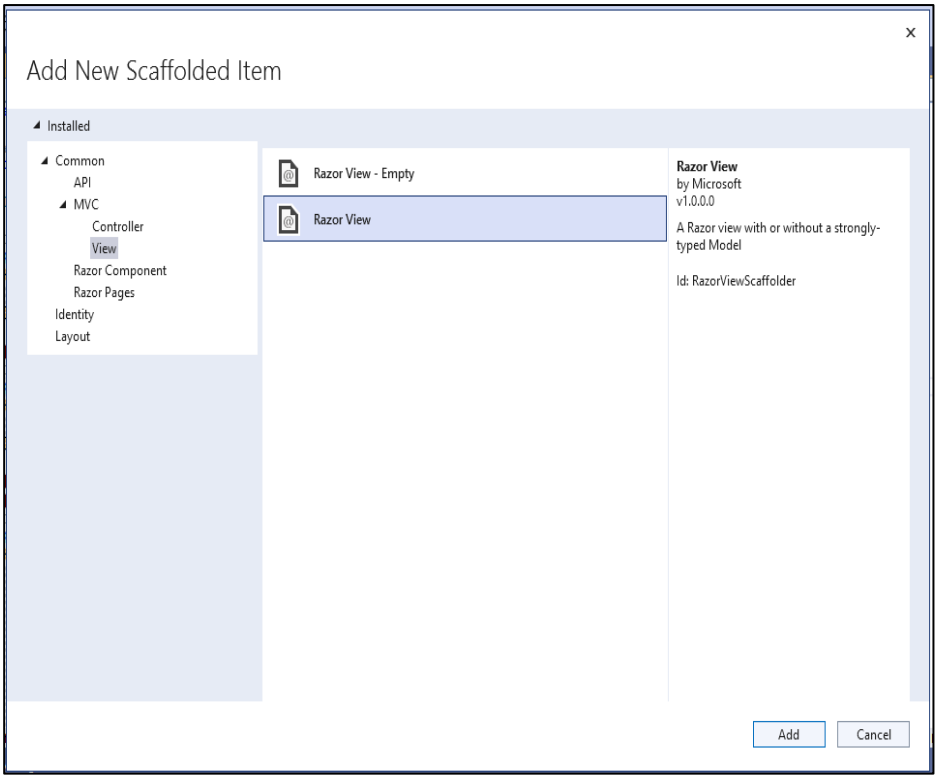
```
        [HttpPost]
        [ValidateAntiForgeryToken]
        0 references
        public ActionResult Craft(Problem model)
        {
            try
            {
                db.Problems.Add(model);
                db.SaveChanges();
                return RedirectToAction("Catalog", "Interrogation");
            }
            catch
            {
                return RedirectToAction("Catalog", "Interrogation");
            }
        }
        0 references
        public ActionResult Catalog()
        {
            return View(db.Problems.ToList());
        }
    }
}
```

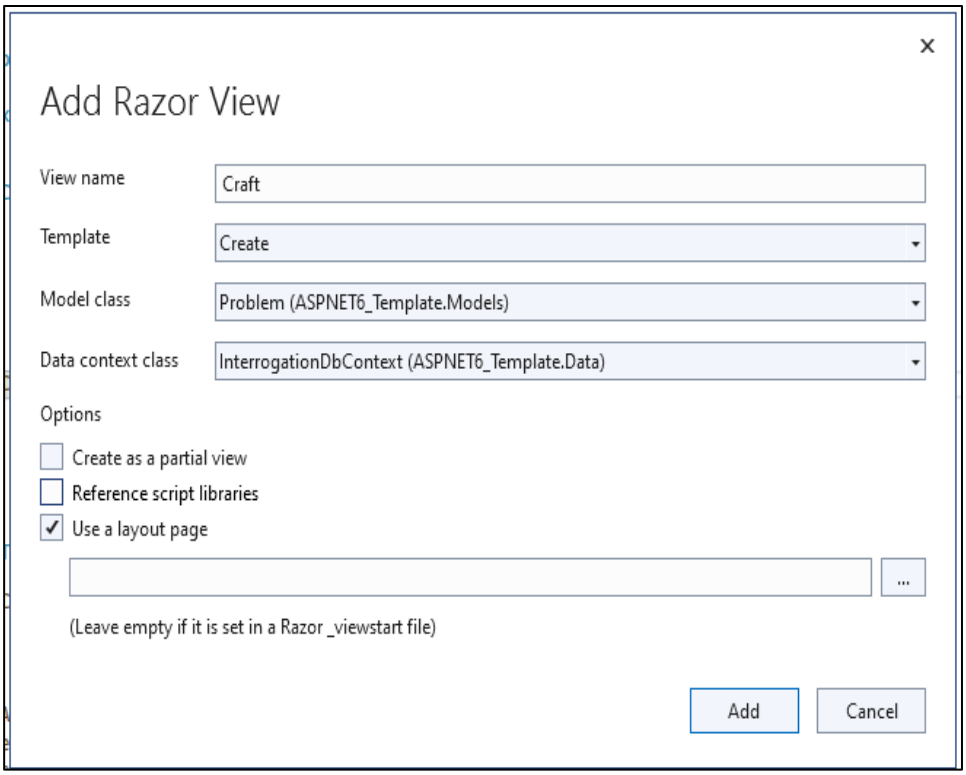
The **Craft** action method gets data from the user and bind those data with the model class **Problem.cs** and save those details in database table named as **Problems**.

After successfully inserting the data in the database the action method should be redirected to another action method named **Catalog**.

The **Catalog** action method should display all the records present in the table **Problems**.

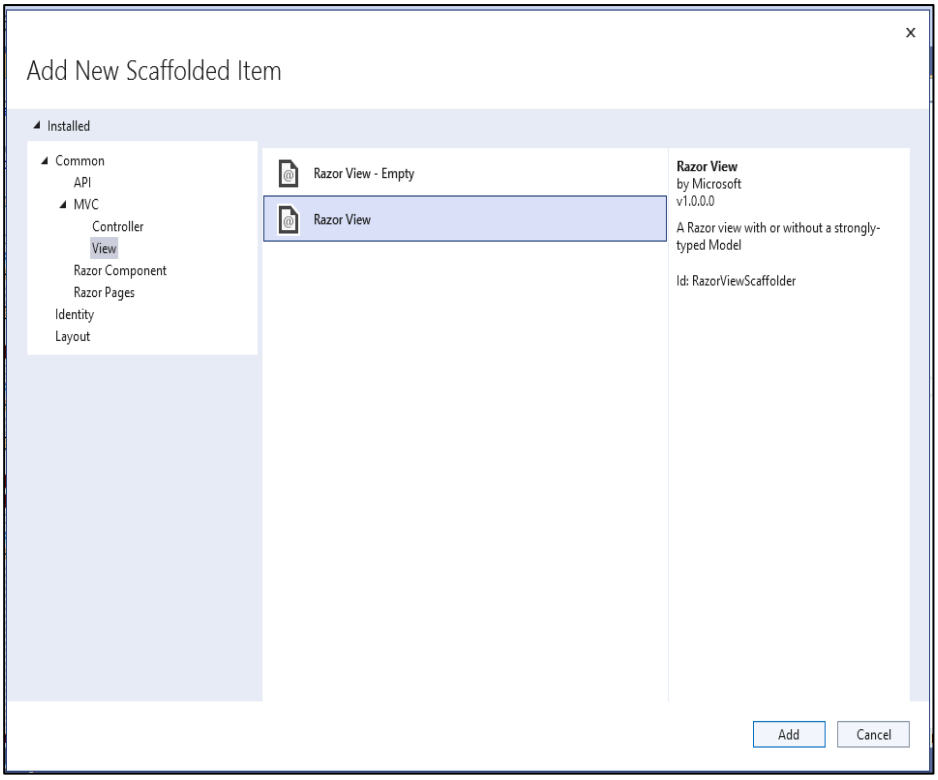
13. Let us now create the View for the action methods named **Craft** and **Catalog**.

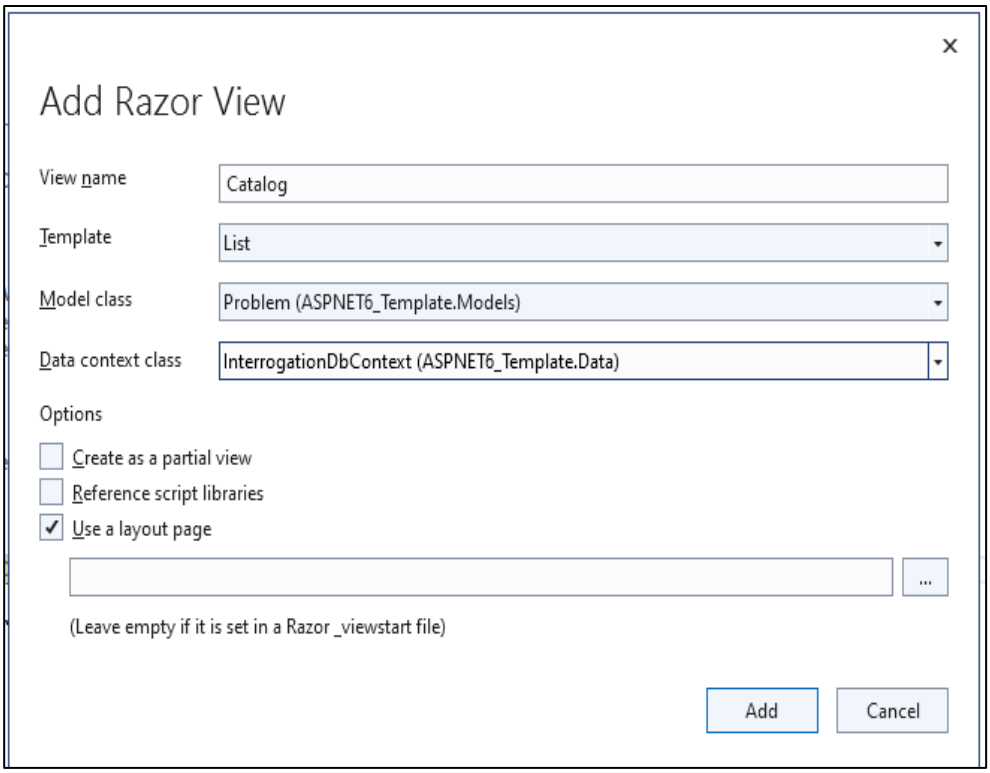

Add New Scaffolded Item
Installed
Common
API
MVC
Controller
View
Razor Component
Razor Pages
Identity
Layout
Razor View - Empty
Razor View
Razor View
by Microsoft
v1.0.0.0
A Razor view with or without a strongly-typed Model
Id: RazorViewScaffolder
Add
Cancel


Add Razor View
View name
Craft
Template
Create
Model class
Problem (ASPNET6_Template.Models)
Data context class
InterrogationDbContext (ASPNET6_Template.Data)
Options
Create as a partial view
Reference script libraries
Use a layout page
...
(Leave empty if it is set in a Razor _viewstart file)
Add
Cancel

```
@model ASPNET6_Template.Models.Problem

@{
    ViewData["Title"] = "Craft";
}

<h1>Craft</h1>

<h4>Problem</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="Craft">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="InterrogatorsProblem" class="control-label"></label>
                <input asp-for="InterrogatorsProblem" class="form-control" />
                <span asp-validation-for="InterrogatorsProblem" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Choice1" class="control-label"></label>
                <input asp-for="Choice1" class="form-control" />
                <span asp-validation-for="Choice1" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Choice2" class="control-label"></label>
                <input asp-for="Choice2" class="form-control" />
                <span asp-validation-for="Choice2" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Choice3" class="control-label"></label>
                <input asp-for="Choice3" class="form-control" />
                <span asp-validation-for="Choice3" class="text-danger"></span>
            </div>
```

```
            <div class="form-group">
                <label asp-for="Choice4" class="control-label"></label>
                <input asp-for="Choice4" class="form-control" />
                <span asp-validation-for="Choice4" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Solution" class="control-label"></label>
                <input asp-for="Solution" class="form-control" />
                <span asp-validation-for="Solution" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" id="btnCraft" value="Create" class="btn btn-primary" />
            </div>
        </form>
    </div>
</div>

<div>
    <a asp-action="Catalog">Back to List</a>
</div>
```


Add New Scaffolded Item
Installed
Common
API
MVC
Controller
View
Razor Component
Razor Pages
Identity
Layout
Razor View - Empty
Razor View
Razor View
by Microsoft
v1.0.0.0
A Razor view with or without a strongly-typed Model
Id: RazorViewScaffolder
Add
Cancel


Add Razor View
View name
Catalog
Template
List
Model class
Problem (ASPNET6_Template.Models)
Data context class
InterrogationDbContext (ASPNET6_Template.Data)
Options
Create as a partial view
Reference script libraries
Use a layout page
...
(Leave empty if it is set in a Razor _viewstart file)
Add
Cancel

```
@model IEnumerable<ASPNET6_Template.Models.Problem>

@{
    ViewData["Title"] = "Catalog";
}

<h1>Catalog</h1>

<p>
    <a asp-action="Craft">Create New</a>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.InterrogatorsProblem)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Choice1)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Choice2)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Choice3)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Choice4)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Solution)
            </th>
            <th></th>
        </tr>
```

```
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model)
        {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.InterrogatorsProblem)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Choice1)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Choice2)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Choice3)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Choice4)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Solution)
            </td>
        </tr>
        }
    </tbody>
</table>
```

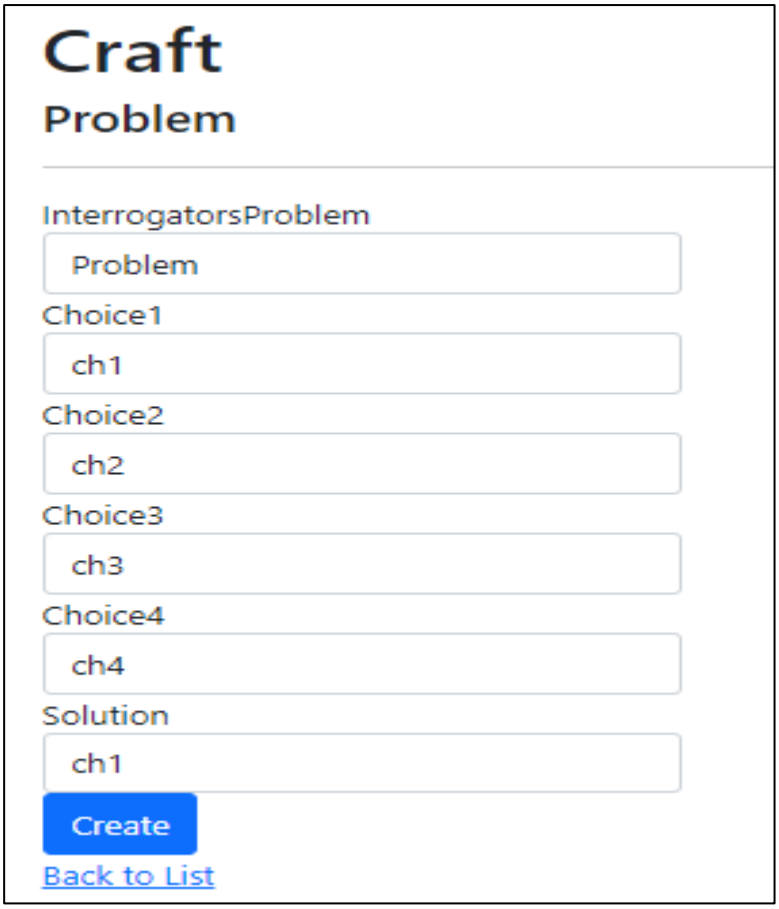
14. Run the application and navigate to the **"Catalog"** page. You should see the form Create New Problem.

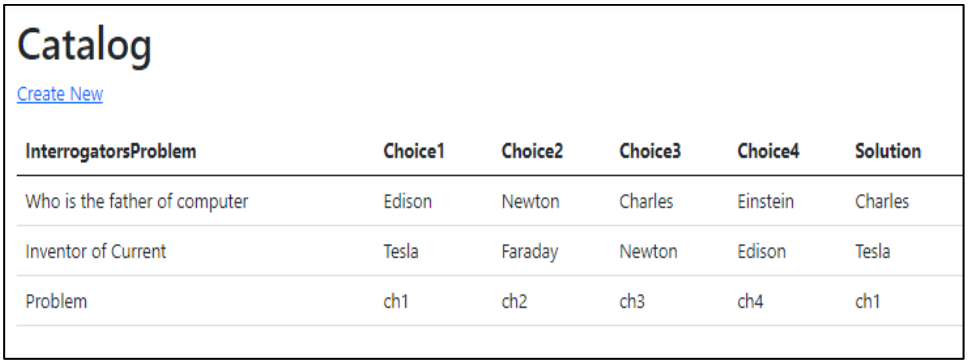
15. Let's see the outcome of the above instructions,

Catalog

Create New

InterrogatorsProblem	Choice1	Choice2	Choice3	Choice4	Solution
Who is the father of computer	Edison	Newton	Charles	Einstein	Charles
Inventor of Current	Tesla	Faraday	Newton	Edison	Tesla





InterrogatorsProblem	Choice1	Choice2	Choice3	Choice4	Solution
Who is the father of computer	Edison	Newton	Charles	Einstein	Charles
Inventor of Current	Tesla	Faraday	Newton	Edison	Tesla
Problem	ch1	ch2	ch3	ch4	ch1

To summarize,

This exercise demonstrates how to store and retrieve values from database using entity framework. We also learnt about model binding by getting the values from the user and setting those values to the properties we declared. Data annotations with custom error messages.